University of Pisa
Master's Degree in Artificial Intelligence and Data Engineering
Distributed Systems and Middleware Technologies

VoltShare

Project Documentation

A distributed coordination system for electric-vehicle charging stations

Group members:

Andrea Innocenti
Lorenzo Barci

Academic Year 2025/2026

Contents

1 Introduction

VoltShare coordinates a network of electric-vehicle charging stations. Drivers find a station, reserve a connector, plug in and are billed; the operator sees the network from a back office. Underneath, the system decides three things: which vehicle gets which connector, how the limited electrical capacity of a site is shared among the cars charging at it, and what happens to both when a node fails.

1.1 Why this domain has real contention

A distributed systems project needs contention that belongs to the problem itself, and never one manufactured for the exercise. This one has it, and the argument is worth making at the start, because everything in the design follows from it.

The operator has no authority over the agents. A delivery platform can assign a courier to an order. Couriers are workers, the system knows where they are, and the dispatch becomes an optimisation problem in which contention disappears. The driver of a private car cannot be assigned anywhere: they have their own destination and can ignore any suggestion. The system can only *arbitrate the requests that drivers make*, and centralising the decision does not remove the contention, because the central decider has no power to decide in the drivers' place.

The exclusivity is physical. One outlet, one vehicle, for thirty or forty minutes. This is not a lock that a cleverer assignment could avoid, and the long occupancy, whose end nobody knows in advance, is what makes the contention observable. A resource held for two seconds produces queueing that disappears into the noise; one held for half an hour produces a driver waiting in a car park.

The distribution is not a didactic device. Charging sites are in different places, and dynamic load management is normally performed by a controller installed at the site, because it has to react quickly and has to keep working when the link to the operator's cloud is down. Our station node is that controller. The requirement that a site keeps charging while cut off from the network, which section 8.3 tests with a real partition, comes from the deployment being shaped like a real one.

1.2 The problems this project solves

Four of them, in the order they cost us work:

- **Exclusive access to a physical resource.** Several drivers ask for the same connector at the same instant, and the outlet must end up promised to one of them.
- **One reservation per vehicle across the whole network.** This is the hard one. Two stations can each be locally correct and still break the guarantee together, so it needs an authority, which then needs replication, election and a quorum to survive its own failure.
- **Sharing a divisible resource.** A site cannot deliver the sum of what its connectors are rated for, so the available power is apportioned among active sessions and renegotiated whenever a car arrives, leaves or fills up.
- **Failures that must not take resources with them.** A node can crash or be cut off, and the two look alike from a distance while behaving differently. Nothing may stay reserved for a party that has gone away.

Section 5 states all seven problems precisely, section 7 gives the mechanism chosen for each, and section 8 covers the failures.

1.3 What is built and what is emulated

The system under development is the coordination backend together with its clients, which in industry terms is a charging-station management system and its site controllers. The hardware cannot be present in a university project: the outlet, its contactor, its meter and the car. That part is an emulator speaking a reduced message set modelled on OCPP 1.6-J, the protocol real charge points use, so the boundary falls where a real interface already exists. Section 4.1.5 makes the split explicit, item by item.

The deployment is seven nodes running as containers on one host: three coordinators, two station controllers, a Tomcat back office and MySQL. Section 10 explains what a single host can and cannot demonstrate, and how a genuine network partition is produced without a second machine.

1.4 A note on the numbers in this document

Measurements quoted here were taken from running deployments and are reported with their date and their conditions, because a figure without either is not evidence. Where something was not measured, the text says so. The same applies to the limits: several of them are declared in section 8.8 rather than left to be discovered, including one hole in the central guarantee that we found by measuring, and closed.

2 Requirements Specification

2.1 Functional requirements

[to be written — 1.5 pages. Account and vehicle, station discovery, reservation, charging session, power allocation, billing and history, notifications. Source: SCOPE §3. Bullet lists are legitimate here: the material is genuinely enumerable.]

2.2 Non-functional requirements

[to be written — 1 page. Exclusive use, no resource lost to a failure, station autonomy, real time by server push, horizontal growth. Source: SCOPE §4.]

3 Domain Rules

[to be written — 1.5 pages. The rules the system enforces, needed to follow the sections that come after: reservation under a lease, no-show and suspension, session authorisation against the reservation, overstay and grace period, power sharing within the site budget, tariffs. Sources: SCOPE §3.3–3.6. Keep it short: these are rules, not discussion. The equivalent of BlackNet's "Game Rules".]

4 System Architecture

4.1 Key design decisions

Five decisions shaped the system, and each is given here with the alternative we discarded and the reason. The sixth and largest, how the network-wide uniqueness of a reservation is enforced, belongs with the problem it answers and is in section 7.2.

4.1.1 The case for middleware

Three of the edges in this system could in principle be written directly on TCP. None of them is, and the reasons differ per edge, which is why we treat them separately instead of adopting one communication technology everywhere.

Between Erlang nodes: distributed Erlang. The stations and the coordinators exchange claims, renewals, announcements and node-failure notifications. On raw sockets each of these would need a framing format, a request-response correlation scheme, a reconnection policy and a way of noticing that a peer has died. Distributed Erlang provides all four: messages are typed terms, a process can be addressed as `{Name, Node}` without knowing an address, and `monitor_node/2` turns a dead peer into a message that arrives in a mailbox like any other. Section 8.2 shows where that last mechanism was not enough on its own and what we added.

Between Java and Erlang: JInterface. The back office has to receive the live station list from the coordinator and to push suspensions to it. The two runtimes share no memory, no type system and no object model, so something has to translate. JInterface lets the JVM join the Erlang cluster as a *hidden node*: it speaks the same distribution protocol, it is addressable by node name, and it exchanges the same terms the Erlang side already sends. The discarded alternative was a REST endpoint on the Tomcat side that the coordinator would call. It would have worked and it would have inverted the direction of control: the cluster would depend on the availability of the web tier to report its own state, and a restart of Tomcat would leave the coordinator retrying HTTP requests instead of doing its job.

Towards browsers and charge points: WebSocket. A charging page is open for the length of a charge, and the power allocated to a car changes every few seconds. The server has to speak first, which HTTP request-response does not allow, and polling at the frequency the data changes would be both late and expensive. Cowboy terminates these connections inside the station node itself, so a state change in a connector process reaches the socket process without leaving the node.

4.1.2 Server-side rendering, with real time only where it moves

The web front end is servlets plus JSP, rendered on the server. The browser holds no application state of its own except in two views, the station page and the live session, which are driven entirely by their WebSocket.

We considered a single-page application, which is the obvious modern choice and is what the reference project for this course used. We set it aside for two reasons. The pages that list stations, sessions and notifications are read-mostly and change on a human timescale, so drawing them in a client application costs more code for a result the servlet already produces. And the course recommends the basic Java web technologies, which is a legitimate constraint on a university project even where a different choice would be defensible in industry.

The consequence to be honest about: server-side rendering cannot push, so anything that has to update by itself needs the WebSocket. We did not spread that across the whole front end. Two pages have it, and they are the two where a value on screen goes stale in seconds.

4.1.3 Coordination separated from the sites

Three coordinator nodes hold the network-wide decisions. Each station node owns its connectors and its electrical budget, and it keeps working when the coordinators are unreachable.

This mirrors how the real industry deploys. Dynamic load management runs on a site controller physically installed at the charging site, because it must react in milliseconds to a car that starts drawing current and must keep the site alive when the link to the operator's cloud is down. What stays central is what can afford a network hop: the registry, the accounts, the invoices. Our station node is that site controller, and the requirement that a site keeps charging while isolated follows from the deployment being realistic in the first place.

The alternative was one central service holding everything, with the stations reduced to hardware adapters. It is simpler to write and it fails badly: an operator whose link goes down would stop delivering power to cars that are physically present and already plugged in. Section 8.3 reports what our split does instead, measured on 3 September with a station cut off from the cluster.

4.1.4 Volatile state in memory, durable state in MySQL

Two kinds of state, in two places, on purpose.

Coordination state (who holds which connector, what each session is drawing right now, how far a battery has come) lives in the memory of the Erlang processes that own it. It changes several times a second per connector, it is worthless once the thing it describes is over, and it can be rebuilt from the hardware on reconnection. Durable state (accounts, vehicles, completed sessions, invoices) lives in MySQL, which is also what the back office reads to draw its pages.

The rule that follows is the one worth defending: **nothing on the interactive path reads the database.** Granting a claim, allocating power, pushing a new state to a browser: none of these issues a SQL query, so a database that is slow or briefly unavailable does not stop cars from charging. The exception is authorising a *new* session, which has to resolve a vehicle to its owner, and section 8.3 describes what happens to an isolated site because of it.

We did not use Mnesia. It would have given us replicated tables inside the Erlang cluster, and the reference project's documentation describes exactly that. Our coordination state is derived from what the stations already hold authoritatively, so replicating it would mean maintaining a second copy of something that can simply be asked for. The argument is in section 8.4.

4.1.5 Where the system stops, and what an emulator stands in for

What we built is the coordination backend and its clients, which in industry terms is a charging-station management system together with its site controllers. What cannot exist in a university project is the hardware: the outlet, its contactor, its energy meter and the car attached to it.

The boundary is placed where a real interface already exists. Charge points talk to management systems over OCPP, an open protocol carried on WebSocket with JSON payloads, whose message set already covers what we need: status notifications, meter values, reservations with an expiry, and charging profiles that cap the power of a connector. Our charge-point channel implements a **reduced message set modelled on OCPP 1.6-J**, and we make no claim of compliance with the full specification.

Table 1: The seven nodes of the delivered deployment.

Node	Technology	Responsibility
Back office	Java, Tomcat, JSP	accounts, authentication, station directory, history, billing; issues the JWT the station verifies
Coordinator ×3	Erlang/OTP	claims, cluster map, node monitoring, election and quorum; feeds the back office
Station ×2	Erlang/OTP, Cowboy	one process per connector, power allocation, the driver and charge-point WebSocket endpoints
MySQL	MySQL 8.0	durable state

What this buys is worth stating plainly: replacing the emulator with real equipment means replacing a client that speaks that protocol. Everything on the coordination side, which is the entire subject of this project, is the logic that would ship.

4.2 Architectural components

The delivered deployment is **seven nodes**, each a container on one host. Table 1 lists them. A single host is a deliberate limit and is discussed in section 10: the course requirement is several *nodes*, and a genuine network partition is produced here by detaching a container from the Docker network.

[to be written — figure: deployment diagram. Seven boxes, the protocol on each edge (HTTP, two WebSocket channels, distributed Erlang, JInterface, JDBC and mysql-otp), the Docker network. Export as PDF from draw.io into figures/ and reference it here.]

Back office (Java, Tomcat). Renders every page server-side and owns everything durable. It authenticates the driver, mints the JWT that the station will verify on its own, prices closed sessions, and applies the no-show suspensions. It also joins the Erlang cluster as a hidden node through JInterface, which is how it learns the live station list without polling the cluster: `StationDirectory` keeps that list in memory and every page read is served from there. A lobby refresh therefore never reaches a coordinator.

Coordinator (Erlang/OTP, three instances). One of them leads and the other two stand by; they are peers of equal capability, and calling them backups would suggest a passive copy of state that does not exist. The leader grants and refuses claims, keeps the map of which stations are up, and pushes station state towards the back office. Its supervision tree uses `rest_for_one` because its children are in dependency order, which section 8.6 explains. The three configured node names are the same on every node (`vs@coord1`, `vs@coord2`, `vs@coord3`), which is what makes both the election and the quorum arithmetic well defined.

Station (Erlang/OTP with Cowboy, two instances). Each station runs one `gen_statem` per connector, a manager that arbitrates them and allocates the site power, and two WebSocket endpoints, one for drivers and one for charge points. The two stations of the delivered deployment are configured differently on purpose:

- `station1`: four connectors rated 150, 150, 150 and 50 kW, on a site budget of 350 kW. The demonstration profile lowers that budget to 200 kW so that the sharing is visible with two cars.
- `station2`: three connectors rated 150, 50 and 50 kW, on 180 kW.

Both sites are oversubscribed, which is the point. Four connectors that can draw 500 kW between them sit behind a 350 kW connection, so the moment three fast chargers are busy the station has to apportion rather than serve, which is P5 in section 7.5.

MySQL. One instance, holding accounts, vehicles, reservations that completed, sessions, invoices and notifications. It is a single point of failure for durable data and we declare it as one in section 8.8, together with the list of things that keep working without it.

The browser is not a node. Pages arrive rendered, and the only state a browser holds is what its WebSocket has just pushed into the two live views. This matters for the failure analysis: there is no client-side cache to reconcile after a reconnection, so a page that comes back gets a complete snapshot and is immediately correct.

5 Synchronization and Coordination Problems

This section states the problems the project set out to solve. The mechanisms that answer them are in section 7, and the ones that answer failures are in section 8; here we say only what is hard and why, because a problem described together with its solution is a problem the reader cannot weigh.

Seven of them are numbered P1 to P7 and are referred to by those names throughout the document.

5.1 The dividing line: one station, or the network

Two resources in this system are exclusive, and telling them apart is what shapes every decision that follows.

Table 2: The two exclusions.

	Object protected	Contended by	Scope
P1	the connector	drivers, among themselves	inside one station
P2	the vehicle	stations, among themselves	across nodes

A connector belongs to exactly one station, always. No other node in the network can grant it, so the station can decide alone and no remote agreement is needed. The vehicle is the only entity in the system that can appear at two stations at the same time: the driver opens two browser tabs and presses *reserve* on both. Each station checks its own connector, finds it free, and grants. Both stations were locally correct, both respected P1, and the network-wide guarantee is broken. Neither of them can detect this alone, because the fact that would reveal it is held by neither.

The connector is stationary and belongs to a node; the vehicle moves and belongs to the network. The first exclusion is settled in-house. The second is not, and it is the reason this project has a coordination problem at all.

One consequence deserves stating early, because it decided which algorithm we could use. The mutual-exclusion algorithms in the syllabus assume a single critical section. Here there is one per vehicle, and they are independent: two reservations for two different vehicles have no reason to wait for each other.

5.2 Problems inside one station

P1: several drivers want the same connector. Two requests for connector 3 arrive in the same millisecond. Both read the connector as free, both proceed, and the outlet ends up promised twice. This is the classic race on a shared resource, and its solution is section 7.1.

The difficulty lies in where the resource lives. A charging connector is a physical object with a cable in it, so what has to be protected goes beyond a row in a database: it is the authority to tell a piece of hardware to deliver power to one car.

P5: the site cannot power everything at once. The sum of what the connectors can deliver exceeds what the site's grid connection can supply, which is true of real installations and is the reason dynamic load management exists. Five cars plugged in at a site rated for three of them is a normal Tuesday, and none of them may be simply refused: they are already connected.

This is a different species of problem from P1, and the pairing is deliberate. A connector is exclusive and is therefore *granted*. Electrical capacity is divisible and is therefore *apportioned*, with the apportionment changing every time a car arrives, leaves, or slows down as its battery fills. Mutual exclusion and permits are two separate chapters of the course material, and this system runs into both inside the same node. See section 7.5.

5.3 Problems between the client and the system

P3: somebody stops answering. A driver reserves a connector and never arrives, or disappears in the middle of a session with the cable still in the socket. Nothing in the protocol can force them back, and the connector cannot stay held for ever waiting for a party that is gone. The same problem in a second form: a station holds a claim at the coordinator and dies, and the coordinator has no way to ask it whether the claim still matters. See section 7.3.

P6: everyone must see the same connector. Two drivers looking at the same station, plus the charge point reporting from the hardware, plus the operator's back office: all of them display a connector whose state changes several times a minute. If each client computes what it believes the state to be, from the last event it happened to receive, they will disagree, and the one who disagrees in the wrong direction drives to an outlet that is already taken. See section 7.6.

P7: a command arrives twice. The driver presses *reserve*, the phone loses signal for two seconds, nothing appears on screen, and they press it again. If both messages reach the station, the naive outcome is two reservations for one car, or a session started twice on the same connector. Retries are not a client bug to be fixed: they are what any reasonable client does on an unreliable link, so the system has to tolerate them. See section 7.3.

5.4 Problems across nodes

These are the problems that would not exist in a single-process version of the same application, and they are the subject of the project.

P2: one vehicle, one reservation, network-wide. The scenario of section 5.1: the same car reserving at two stations at once, with each station locally correct. Whatever solves it must place a decision somewhere that both stations consult before committing, and it must do so without turning every reservation into a network-wide serialisation of every other. See section 7.2.

P2b: the authority itself fails. Once P2 is answered by an authority, that authority becomes the thing that can fail. Two failures, with different consequences. If it crashes, nobody can reserve until something replaces it. If a partition splits the network, both sides may conclude that the other is gone and each appoint its own authority, and two authorities granting claims for the same vehicle break exactly the guarantee the authority exists to provide. Split brain is worse than downtime here, because downtime refuses and split brain grants. See section 7.4.

P4: a station node goes away. It can crash, or its uplink can be cut. Both look similar from a distance and they are different failures.

A crashed node takes its sessions with it: the process metering them is gone and no row reaches the database. A partitioned node keeps delivering power to the cars plugged into it while the rest of the network stops hearing from it, so the site works and the operator goes blind. In both cases the vehicles that station was holding are stranded: their claims are alive in the coordinator, and their drivers cannot reserve anywhere else until those claims expire. See section 8.3.

Detecting any of this is itself a problem. A failure that is not noticed is a failure that is not handled, and the detector the runtime gives us for free does not cover both cases. Erlang delivers `nodedown` when a TCP connection breaks, which is what a killed container looks like. A node that stops answering *while its socket stays open*, which is what a partition looks like, remains invisible until the distribution's own tick expires, and `net_ticktime` defaults to sixty seconds. Sixty seconds is a long time for a coordinator to keep granting claims it has no right to grant. Section 8.2 gives the measurements and the answer.

5.5 Communication problems

Three of them, and together they are the reason this system needs middleware at all. Section 4.1 argues that point in full; here we state what has to be carried.

- **Heterogeneous runtimes exchanging structured data.** Authentication, the station directory, billing and the web front end live in a JVM; coordination and the connectors live in Erlang. The two halves have to exchange typed messages, and they run in processes that share no memory, no language and no object model.
- **Long-lived, low-latency connections towards many clients.** A charging page is watched for the length of a charge, which is half an hour or more, while power allocations change every few seconds. Polling would be both late and wasteful, so the connection has to stay open and the server has to be able to speak first.
- **Asynchronous propagation between nodes.** The back office draws its station list from what the cluster knows, and the cluster changes when nobody is looking at it. Asking on every page view would put the operator's web traffic on the critical path of the coordinator; the cluster therefore has to push, and the back office has to be reachable as a peer.

6 Architecture Protocols

[to be written — 1.5 pages. Which protocol runs on which edge and why that one, split into client-facing and internal as BlackNet does. HTTP browser to back office; WebSocket browser to station; WebSocket/JSON charge point to station; distributed Erlang station to coordinator;

JInterface back office to coordinator; SQL to MySQL. Overview only: the message sets are specified in section 9. Sources: SCOPE §6, contracts/.]

7 Addressing the Problems

Section 5 listed the problems. This section describes the mechanism chosen for each one, the alternatives that were set aside, and what each choice costs. Failures have a section of their own (section 8), because there are five different mechanisms answering them, one per failure.

Two of the problems are settled inside a single station and two of them are not, and the difference decides everything else. A connector belongs to exactly one station, always. No other node can grant it, so no remote agreement is needed and the station can decide alone. A vehicle is the only entity that can appear at two stations at the same time: the driver opens two browser tabs and presses *reserve* on both. Each station checks its own connector, finds it free, and grants. Both are locally correct, both respected the exclusivity of their own outlet, and the network-wide invariant is broken. Neither station can detect this by itself, because the missing fact is held by neither of them.

7.1 Mutual exclusion on a connector (P1)

Each physical connector is owned by one Erlang process, a `gen_statem` implemented in `vs_connector`. Every request that touches that outlet becomes a message in that process's mailbox, and messages are handled one at a time. Two drivers reserving the same connector in the same millisecond are therefore serialised by the runtime, and the second one finds the connector already in state `held`.

There is no lock in the station code, and none is needed. This is the actor model applied to its textbook case: a resource paired with the process that guards it, so that mutual exclusion becomes a property of the mailbox instead of a discipline the programmer has to keep.

The state machine carries a second guarantee that is easy to overlook. A session cannot start on a connector held by somebody else, and no check anywhere in the code is what prevents it: `charging` is simply unreachable from `held` with the wrong vehicle. There is no branch to forget. Events that do not apply to the current state fail to match, and the connector answers with a refusal that costs one clause. Enforcement of this kind fails loudly during development, where a missing clause is a crash and therefore cheap.

7.2 One reservation per vehicle, network-wide (P2)

This is the guarantee the architecture is built around, and the mechanism is a *claim*: a permission, granted by the coordinator, recording that a given vehicle is now committed and to which station.

The two words matter throughout this document. A **reservation** is what the driver sees: connector 3 at this station, held for me until 18:07, owned by the connector process. A **claim** is what made it possible: an entry at the coordinator, about a vehicle, valid across the whole network. The reservation is local and concerns an outlet; the claim is global and concerns a car.

7.2.1 Claim first, commit after

A station obtains the claim *before* it creates the reservation, never the other way round. The order is the whole mechanism, and it is worth stating why.

In the wrong order, a crash between the two steps leaves a reservation that no other node knows about. The vehicle can then obtain a second reservation elsewhere, and the invariant

is broken silently, with nothing in the system aware of it. In the right order, the same crash leaves a claim that was granted and never used. Nobody can reserve for that vehicle for a few minutes, and then the claim expires on its own. The system errs towards refusing rather than towards granting twice.

This is the reasoning behind a write-ahead log, applied to a permission instead of to a balance: record the commitment before acting on it, so that the recovery path only ever has to undo an intention.

7.2.2 Why a central arbiter

The coordinator is a single process, `vs_coord_srv`, holding the claim table in its own memory. Concurrent requests from different stations queue in one mailbox and are decided one at a time, which is what makes the decision serialisable at all. In the vocabulary of the course this is centralised distributed mutual exclusion, with the difference that the critical section is not one: there is one per vehicle, and they are independent.

That last point decided the algorithm. We considered four designs.

One coordinator, unreplicated. The simplest, and its failure is only partial: the coordinator sits outside the path that delivers power, so sessions in progress survive it. We rejected it because it leaves an avoidable single point of failure on the one guarantee the system exists to provide.

A `UNIQUE` constraint in MySQL. A uniqueness constraint on the vehicle would enforce P2 correctly and cost almost nothing to write. Two reasons against it. The coordinator has to exist anyway, for the cluster map and for node monitoring, so this option removes no component. And a claim that *expires by itself* is a timer inside a process, whereas in the database it becomes either a cleanup job or a filter on every read, which is polling: the one thing this architecture avoids on every other edge. It also moves the single point of failure onto the database instead of removing it.

Direct coordination between stations (Ricart-Agrawala). No central authority by construction, which is attractive. It costs $2(n-1)$ messages per reservation, needs careful handling of nodes that die holding a token, and, decisively, it serialises access to *one* critical section. Here every vehicle is an independent one, and two reservations for two different vehicles have no reason to wait for each other.

Partitioning the vehicle space across coordinators. Technically the cleanest of the four: the node responsible for $hash(vehicle)$ decides, no node sees all the traffic, and a failure blocks one shard instead of the network. We set it aside because every node must agree on the same partition map, the map must be rebalanced when membership changes, and each shard still needs its own failover. It adds to the chosen design rather than replacing it, and it is recorded as future work in section 12.

7.2.3 Ordering decided by one clock

When a claim is granted, the coordinator stamps it with `granted_at` and sends that timestamp back with the grant. The station stores it and echoes it in every renewal.

The purpose is conflict resolution after a failover. If two claims for the same vehicle ever meet in the table, the older one wins, and the comparison is between two timestamps produced by the *same* process. No two machines with unsynchronised clocks are ever compared. We did

not need logical clocks to get this: the coordinator already serialises the decisions that matter, so a single physical clock, used at one place only, is sufficient and simpler to defend.

7.3 Leases, and commands that arrive twice (P3, P7)

A driver who reserves and never arrives is not a failure of the system, and it cannot be solved by asking the client to be well behaved. The answer is a lease: every reservation is granted with an expiry, and when the expiry passes the connector returns to `free` with no operator action and no cooperation from the party that disappeared.

There are two nested expiries, and they protect against two different absent parties:

- the **reservation lease** protects against the driver who never shows up. It is the timer the connector process holds;
- the **claim expiry** protects against the station that dies without releasing what it holds. Without it, a crashed node would lock its vehicles out of the entire network until somebody intervened.

The claim outlives the reservation on purpose, by `CLAIM_GRACE_SECONDS`, so that a station renewing slightly late does not lose a claim for a car that is charging. Stations refresh what they hold every `CLAIM_RENEW_INTERVAL_MS`, ten seconds by default. The default lease is fifteen minutes; the demonstration profile in `.env.demo` shortens it to 90 seconds so that a no-show can be watched from beginning to end without waiting a quarter of an hour. No clock is scaled: only the parameters change.

This is the leasing pattern of the course material, the same mechanism used by distributed garbage collectors to decide that a remote reference has gone away. The property that makes it work here is that the resource is released by the passage of time and not by a message, so no participant can hold it hostage by falling silent.

At-most-once for driver commands. A driver on a mobile connection sends `reserve`, sees nothing happen, and presses the button again. If both messages arrive, the naive outcome is two reservations, or a session started twice.

Every driver command therefore carries a mandatory `request_id`, and `vs_driver_proto` keeps a small cache of recent identifiers with the frames that answered them. A repeated identifier is answered with the stored reply and is never dispatched a second time. The retry is invisible to the connector, which is the point: the deduplication belongs at the edge, where the duplicate is created. The state machine never has to reason about it.

7.4 Coordinator replication: election and quorum (P2b)

Making the coordinator authoritative creates the problem of the coordinator itself failing. Three coordinator nodes run; one is the leader and is the only node that grants or refuses claims. A station that addresses one of the other two is redirected to the leader.

Election. When the leader stops responding, the survivors elect a new one with the bully algorithm over their fixed node identifiers: the highest live identifier wins. A node that notices the leader is gone sends `ELECTION` to the nodes above it. Silence means it is the most senior node alive and it announces `LEADER` downwards; an answer means somebody more senior is handling it, and it waits. With three nodes this settles in two rounds of messages. Detection is by timeout, which means we assume a *synchronous* system, an upper bound on response time. That assumption is stated here because the correctness of the detection depends on it, and it is the kind of thing an examiner is right to ask about.

Quorum. Bully alone does not prevent split brain. In a partition each side happily elects its own local maximum, and two authorities granting claims break precisely the invariant the coordinator exists to protect. So a coordinator serves only while it can see a majority of the configured coordinator cluster, itself included: two out of three. A node in the minority suspends itself and refuses everything, leader or not.

This is why the cluster has three members and not two. With an even number, a partition can produce two halves that each consider themselves entitled to grant.

The cost is real, and we state it: during a partition, the minority side refuses *new reservations*. Charging sessions already running are untouched, because the coordinator is not on the path that delivers power. It only decides who may hold a vehicle. Refusing to serve when the invariant cannot be guaranteed is deliberate, and it is the same instinct that runs through the lease and the claim expiry: when in doubt, withhold.

State after a failover. A new leader inherits nothing, and it does not inherit a replicated log either. It asks the stations what they hold and rebuilds the claim table from their answers. The reason this is possible, and the reason it is a design decision rather than a shortcut, is developed in section 8.4: the coordinator was built as an index over state that lives somewhere else.

7.5 Sharing the site power (P5)

The last problem is a different species from the others, and putting it next to them is deliberate. A connector is exclusive, so it is granted to one holder. The electrical capacity of a site is divisible, so it is not granted at all: it is apportioned, and the apportionment changes every time a car arrives, leaves, or changes what it can absorb. Mutual exclusion and permits are two separate chapters of the course, and this system needs both, inside the same node.

The site budget is lower than the sum of the nominal ratings of the connectors, which is true of real installations and is what makes the problem exist. `vs_power` recomputes the split on every arrival, every departure, and on a periodic tick that follows the charging curve as batteries fill.

The policy is max-min fair share with hand-back. Every active session starts from $budget/N$. A session that cannot absorb its share, either because the car is small or because it is tapering near full, takes only what it asks for and hands the surplus back, which is then split again among the others. The loop repeats until nobody hands anything back or the budget is exhausted.

We chose it over the two alternatives named in the requirements for what it yields and for how it can be defended. Proportional-to-demand does not deliver more energy: it saturates the same budget and only changes who receives what, so it has to be argued as a political choice about which cars matter more. Reservation-holders-first would starve a walk-in session down to suspension, which contradicts the rule that a no-show costs a driver the right to book while leaving them free to charge. Fair share with hand-back is one sentence long, and no kilowatt sits unused while somebody could absorb it.

The floor. Below a minimum viable power a session is suspended rather than starved, which the requirements ask for and which `vs_power` implements by allocating exactly 0.0 kW: the session stays open and draws nothing. The condition has two halves. A session is suspended when its share is under `MIN_CHARGE_KW` *and* under what it asked for. The floor exists to answer scarcity, so a car that receives exactly what it requested is not being starved by anybody, and taking it to zero would free nothing for anybody else.

When the budget really is short, the session suspended is the most recent arrival, ordered by `started_at` with the connector identifier breaking a tie. That rule is stable by construction,

because `started_at` never changes. An earlier version ranked sessions by how far they were from their demand instead, and produced a two-tick oscillation: suspending a session zeroed its limit, a zero limit made it look demand-bound, and the next tick reversed the decision. The lesson generalises beyond this project. A control rule must not take as input a quantity that the rule itself sets.

7.6 One source of truth for every client (P6)

Several parties look at the same connector: the driver who holds it, other drivers deciding where to go, the charge point reporting from the hardware, and the operator's back office. They must agree, and the cheapest way to make them agree is to give them nothing to compute.

The station holds the authoritative state and pushes a *complete* snapshot of it, built by `vs_connector:build_snapshot/2`. A client never derives a new state from an event it received: it replaces what it is displaying with what has just arrived. A page that misses a message is therefore wrong only until the next push, and a page that reconnects is correct immediately, with no replay and no reconciliation. Snapshots go out on every change and on a periodic tick, so a browser that was asleep converges without asking.

This is also why the at-most-once cache of section 7.3 deliberately excludes one message. A repeated command must be answered with its stored reply, but the station snapshot must never be replayed from a cache: by the time a duplicate arrives, the stored snapshot describes a connector that has moved on, and answering with it would put a stale state on a page that had the current one. The duplicate is answered, and the state comes from the live push.

8 Fault Tolerance and Recovery

Fault tolerance in this system is not one mechanism. It is five, chosen per failure, and the claim worth defending is that each was picked for a specific reason. None of them was applied across the board.

One idea organises them. Every component keeps only the state it owns, and every other component treats that state as recoverable from its owner. Tolerance follows from ownership rather than from replication, which is why there is no consensus protocol anywhere in this project. Its absence was decided, and section 8.4 gives the argument.

8.1 The failure model

We assume crash-stop and network partition. Byzantine behaviour is out of scope: a node that answers, answers truthfully. Table 3 lists what can fail and what the system does about it.

8.2 Two failure detectors, because there are two failures

This is the design decision in this section most worth defending.

Erlang gives us `net_kernel:monitor_nodes/1` for free, and it fires the instant a TCP connection breaks. That is exactly what a killed container looks like. It is immediate and it costs nothing.

But a node that stops answering *without* closing its socket is invisible to it. A network partition, a frozen virtual machine, a host that has started swapping: the connection is still open, so no `nodedown` is delivered until the distribution's own tick expires, and `net_ticktime` defaults to sixty seconds. Sixty seconds of a minority leader still granting claims is precisely the failure the quorum of section 7.4 exists to prevent.

Liveness is therefore decided by an explicit heartbeat, implemented in `vs_coord_membership`. Each coordinator announces itself to its peers once a second, and a peer that misses three

Table 3: What can fail, how it is noticed, and what it costs.

What fails	How it is noticed	What happens	What is lost
Coordinator, crash	`nodedown`, at once	election; the new leader rebuilds from the stations	nothing
Coordinator, partition	heartbeat, 3 s	the minority suspends itself	new reservations, minority side
Station node, crash	`monitor_node`, at once	the coordinator drops that station's claims	sessions running there
Station uplink, partition	`monitor_node`, on the tick	the site keeps charging, starts nothing new	the operator's view
Charge point	socket closes, then 30 s grace	session closed with the last measured energy	nothing measured
Back office	nothing, it is a hidden node	the cluster is unaffected; it re-reads on restart	nothing durable
MySQL	connection errors	not tolerated, see section 8.8	writes, until it returns

announcements in a row is treated as gone. Three seconds to detect a partition instead of sixty. `monitor_nodes` is subscribed to as well, but only to react faster when it does fire: it can never conclude anything the heartbeat would not have concluded a moment later.

Two details of the implementation are worth a sentence each. The heartbeat is sent with a plain asynchronous send to a registered name, which never blocks and never fails on an unreachable node, so no dead peer can slow down the tick that is supposed to notice it. And the state starts optimistic, with every peer assumed alive: starting pessimistic would mean that after a cluster-wide restart every node believes itself alone for three seconds, and three coordinators would elect themselves at the same moment.

Measured on 1 September, isolating the leader with `docker network disconnect`:

- `QUORUM LOST (1 of 3) ... abdicating` at **2.12 s**;
- a new leader serving with both claims at **2.29 s**;
- the distribution's own `nodedown` at **64.6 s**.

That last number is the argument. Without the heartbeat the system would have been incorrect for a full minute, and no test would have revealed it, because `docker kill`, the obvious way to simulate a failure, closes the socket and never exercises this path at all. Two kinds of failure need two experiments, which is why the demonstration shows both `kill` and `disconnect`.

8.3 A station dies, and a station goes silent

There are two distinct failures here, and conflating them is the most visible mistake available in this part of the system.

The node dies. `vs_coord_srv` takes an `erlang:monitor_node/2` on every station it has heard from, and on `nodedown` it erases that station's claims.

Doing nothing here would be the intuitive choice and it would be wrong. A claim is a promise that a vehicle is committed *somewhere*. If the somewhere no longer exists, the promise locks the driver out of the whole network until the lease expires: up to fifteen minutes of being unable to reserve anywhere, because of a failure they did not cause. Dropping the claims turns a station failure into a local one.

We state the cost. Charging sessions on that station are lost, because the process that was metering them is gone and no row was written. A *restart* does preserve the energy figure, because the charge point is the side that counts it and brings its running total back in the message it re-sends on reconnect. What comes back is the measurement. The session itself is over.

The uplink is cut. The more interesting failure, and the one with a plain meaning outside a classroom, is a station that is alive and unreachable. The car park is delivering power and the operator cannot see it.

Measured on 3 September, with `station2` isolated by `docker network disconnect`. The cars kept charging: energy climbed inside the isolated station from 0.248 to 0.403 to 0.558 kWh on one connector, and from 14.069 to 14.208 kWh on another. The coordinator logged `Node vs@station2 not responding`, dropped that station and its claims, and the lobby emptied, because the lobby is drawn from what the coordinator pushes. On reconnection the station announced itself again and neither session had been interrupted.

Two details sharpen the picture. The published ports survive, so port 9102 still answered `426 Upgrade Required` throughout: a driver physically at the site could still open its page and use it. What is lost is the operator's view, which is the honest shape of this failure and is exactly why reservations expire on their own instead of waiting for somebody to cancel them.

But no new session can start. Authorising a car means resolving vehicle to owner, and that is a MySQL query across the severed link. `vs_cp_proto` logs `no account for vehicle N` and opens nothing, while charges already running are untouched, because they never touch the database. An isolated site finishes what it started and accepts nobody new. A real site controller with an unknown card and no uplink behaves the same way.

8.4 Rebuilding the claim table after a failover

A new leader inherits nothing. The claims were granted by its predecessor and lived in that process's memory, which died with it, so before granting anything it has to find out which vehicles are already held.

It does so by asking. The coordinator sends `who_do_you_hold` to every connected node that is not a coordinator, and each station walks its connectors and replies with those in `held` or `charging`, from memory. The leader switches from `rebuilding` to `serving` once the answers are in. Requests received in the meantime are refused with a retry indication, which the stations map onto a temporary refusal. The driver waits a second and then gets their connector.

Why asking is enough. This is the decision the whole failover story rests on. **The coordinator does not own the state it holds; it is an index over it.** The authoritative copy of "connector 3 is held for vehicle 88 until 15:42" lives on the station, which is also the only party that can act on it. The coordinator holds a derived view whose only purpose is to make the uniqueness check of section 7.2 a local lookup, and a derived view can be rebuilt from its sources.

That is why there is no replicated log, no consensus on a state machine, and no persistence at the coordinator. Replicating the claims would mean maintaining a second copy of something the stations already hold authoritatively: more machinery, more ways for two copies to disagree, and nothing gained. If the coordinator held information derivable from nowhere else, a replicated log would be unavoidable. It was designed so that it does not, and that is worth stating as a choice, because it was one.

Two paths converge. The explicit query is the fast path. There is a slower one that works even when the query reaches nobody: renewals arrive every ten seconds carrying the claim identifier and its original grant timestamp, and a renewal for an unknown claim is adopted rather than refused. The system would converge on its own within one renewal interval. The query makes recovery take a second instead of ten.

A hole we measured, and closed. The two paths were not enough on their own, and the defect is instructive because neither number involved was wrong by itself.

A coordinator that rejoins after a partition elects itself before its connections to the stations have been re-established. Its rebuild then asks nobody, receives nothing, and concludes in a microsecond that the network holds no claims. On 1 September a vehicle obtained a *second* reservation at another station 272 ms after such a leader began serving, and the invariant stayed broken for 13.65 s until the first renewal re-presented the older claim. The defence in place waited 2000 ms for stations to speak, while renewals arrive every 10000 ms: it covered a fifth of a cycle. The rebuild window is tuned to failure detection and the renewal interval is tuned to the cost of traffic at rest. The defect was that the first was being used to defend against the second.

The fix has two halves, one on each side of the system. Stations now re-announce and renew immediately on `nodeup` instead of waiting for their next tick, which puts their claims in flight within a second of the network coming back. And a rebuild that returns *no stations* while the table is *also empty* no longer starts serving. That combination does not describe an idle network: it is the signature of a leader that could not ask. The coordinator stays in `rebuilding` until a renewal tells it that the world exists.

The residual cost is declared. A rejoining leader refuses new reservations until it has heard from one station, which with the immediate renewal is a matter of milliseconds. The deadline timer is deliberately not cleared during that wait: it is the ceiling. If no renewal ever arrives, the backstop fires and the coordinator serves with a warning, because one that is stuck for ever is worse than one that is briefly wrong.

8.5 A charge point dies: grace, then honesty

The connector does not react to a lost socket immediately. A socket blip is not a broken charger, and a car charging perfectly well should not be stopped by our own reconnection logic. The `DOWN` arms a grace timer instead.

The window is one budget of ninety seconds, split in two: sixty seconds as the socket's idle timeout, which is two missed heartbeats at the thirty-second interval the charge-point protocol specifies, and the last thirty seconds as the connector's own grace. Splitting one budget in two places is a source of bugs and was one: an earlier version let both sides start a full window, so three missed heartbeats became six, and a charge point that had gone quiet stayed reservable for three minutes.

If the grace expires mid-session, the session is closed with the last energy the charge point reported. That figure is kept because the charge point was the only side counting, so its last report is the best available truth. The connector then becomes `out_of_service`. It does not go back to `free`: an outlet with no hardware attached cannot serve anyone, and offering it would send a driver to a socket that will do nothing when they arrive.

Recovery needs no operator. A charge point that boots and reports itself available lifts the connector back to `free`.

8.6 Supervision inside a node

Each supervision strategy encodes a claim about dependencies between processes, and the three supervisors in this system make three different claims. Choosing between them is a design decision of the same kind as the ones above.

- `vs_coord_sup` uses `rest_for_one`. The children are started in dependency order, so restarting one restarts everything downstream of it. When the process holding the claim table dies, everything derived from it must die too: a survivor holding references into a table that has been recreated is worse than a clean restart.
- `vs_station_sup` uses `one_for_one`, because its children are genuinely independent. The station manager crashing must not take down connectors that are delivering power to real cars.
- `vs_connector_sup` uses `simple_one_for_one` with `restart => transient`. Every child is the same kind of process, one per connector, and a connector that terminates normally stays terminated.

Two defects found while reviewing this code show that supervision only works if the rest of the code cooperates, and both were failures of supervision rather than of logic. The rebuild worker was started with `spawn_link` while the coordinator does not trap exits, so any exception inside it, including a malformed reply from any station on the network, would have killed the process holding every claim, and `rest_for_one` would then have taken the whole subtree down with it. It is now `spawn_monitor`, which delivers the same news without the lethality. Separately, the `rebuilding` state had exactly one exit, the success message: a worker that died before sending it left the coordinator refusing every reservation in the network, for ever, in silence. There is now a `DOWN` handler and a deadline, and both fall through to `serving` with a warning.

The general lesson is worth stating, because it applies well beyond this project. A supervision tree protects you from the failures you routed through it. A linked process and a state with no timeout are both ways of routing a failure around the tree.

8.7 Three delivery guarantees on one wire

The bridge from the coordinator to the back office carries three kinds of event, and they deliberately do not get the same guarantee, because the cost of a duplicate is different for each.

- **The session row: at-least-once over an idempotent write.** The back office prices a session with `UPDATE sessions SET cost_cents = ? WHERE id = ? AND cost_cents IS NULL`. A repeat updates zero rows and is not an error. At-least-once over an idempotent write is the standard way of obtaining effectively-once, and billing is also driven by a periodic sweep, so a lost message delays the invoice without changing what it says.
- **The no-show strike: at-most-once.** A counter cannot recognise a duplicate, and a doubled strike would suspend an innocent driver. One send, no retry. A lost strike is a smaller wrong than an invented one.
- **The notification: at-most-once.** A duplicate is a row somebody reads twice; a loss is a row they never see. Neither is good, and we chose the quieter failure.

Choosing the guarantee per message rather than per channel is the point. A single reliable-messaging layer would have forced one answer onto three questions that have different answers.

8.8 What tolerates nothing, and is declared

Three things in this system have no recovery path, and we prefer to name them than to let them be discovered.

MySQL is a single point of failure for durable data. Replicating it was out of reach for this project. The honest position is not that every single point of failure has been eliminated, which is not true of any system this size, but that we chose which ones to keep and know what happens when they fail. Note what does *not* depend on the database: charging continues, power sharing continues, and the coordinator keeps granting claims, because none of them reads MySQL on the interactive path.

Sessions on a dead station are lost, as section 8.3 describes. The process that was metering them is gone.

The rebuild can still miss a station that is unreachable at exactly the wrong moment. Its claims are absent from the new leader's table, so in principle one of its vehicles could obtain a second claim elsewhere before the conflict is noticed. In practice a car plugged in at that station is not in another city, and the conflict is resolved when the station returns and re-presents the older claim, which wins. The window is narrow and it is real, and it is declared here instead of being left for somebody to find.

9 API and Protocol Specification

Five channels carry everything in this system, and section 6 said which runs where and why. This section specifies what travels on each one.

The three channels between our own components were written as contracts before either side was implemented, and they live in the repository as `contracts/*.md`. Two of them are marked *frozen*: changing one requires a review by both developers, because each side is written by a different person against the text and not against the other's code.

9.1 HTTP: browser to back office

Nine servlets, mapped with `@WebServlet` annotations and no deployment descriptor. Everything is rendered server-side and forwarded to a JSP; there is no JSON API on this channel, because the browser receives pages.

Table 4: HTTP endpoints of the back office. The last column marks the ones behind `AuthFilter`.

Path	Method	Effect	Auth
`/register`	POST	creates the account and its vehicle profile	
`/login`	POST	authenticates, opens the `HttpSession`, mints the JWT	
`/logout`	GET	invalidates the session	
`/stations`	GET	the directory, read from `StationDirectory` in memory	•
`/station`	GET	one station's page, with the JWT and the WebSocket URL	•
`/session`	GET	the live session page for a connector	•
`/profile`	GET	account and vehicle	•
`/history`	GET	past sessions with their invoices	•
`/notifications`	GET	notifications stored for this driver	•

9.1.1 The token the station will trust

A station has to recognise a driver without asking the back office. That is what keeps a site working while Tomcat is down, and what keeps driver traffic off the Java container. The token is the only thing the two sides share.

It is a JWT signed with **HS256** using a symmetric secret, passed to both sides in the environment as `VOLTSHARE_JWT_SECRET` and at least 32 characters long, because HS256 needs 256 bits of key material and the Java library refuses to sign with less. It lives 60 minutes. Java signs with `jjwt`, Erlang verifies with `jose`.

```
{ "sub": "12", "username": "andrea", "vehicle_id": 88,
  "iss": "voltshare-backoffice", "exp": 1755792000 }
```

Listing 1: The claims, plus the standard `iat`.

`sub` is the user id as a string, which the JWT specification requires, and `vehicle_id` is a number. The station needs both: the first attributes the session to an account, the second matches the claim, which is per vehicle.

On the first frame of the WebSocket, which must arrive within five seconds of the connection opening, the station verifies the signature, checks that `iss` is exactly `voltshare-backoffice`, checks that `exp` is in the future, and extracts the two identifiers. A failure closes the socket with `4401`, an expired token with `4408`. The station never calls the back office about a token, whether it accepts one or throws it out.

There is a trap here that the library invites, and `contracts/sample-tokens.md` records it: `jose_jwt:verify/2` checks *only the signature*. Issuer and expiry are ordinary claims, so a token that verifies is not yet a token to accept, and both checks are written out explicitly on the station side.

How the token reaches the browser. It never passes through JavaScript storage. The servlet puts it in the session, the JSP renders it into a hidden element as a `data-` attribute through `<c:out>`, and the WebSocket code reads it from there.

The shape matters for a reason that is not obvious. An earlier version declared these values as JavaScript constants inside a `<script>` block, which meant the JSP was building JavaScript source out of EL. Only the token is the back office's own: the WebSocket URL comes from a station node, travels in its `station_up` announcement through the coordinator, and is republished by `StationDirectory`. A station announcing itself with a URL containing a quote and a semicolon would have closed the string and run its own script on every driver's page. An attribute rendered through `<c:out>` cannot become code, whatever it contains, because it never reaches a JavaScript parser.

9.2 WebSocket: driver to station

[to be written — 1 page, from contracts/ws-driver.md. The handshake and the mandatory request_id, then one table of client-to-server actions (join, reserve, cancel_reservation, stop_session) and one of server-to-client frames (the station snapshot of 5.1, the live session of 5.2, errors, the final closed). Say which frames are cacheable for the at-most-once replay and which are never replayed, and why: section 7.6 gives the reason.]

9.3 WebSocket: charge point to station

[to be written — 1 page, from contracts/ws-chargepoint.md. The reduced OCPP 1.6-J message set: boot, status notification, meter values, the power-limit command. State which OCPP

Table 5: The claim protocol. `ReqId` is generated by the station and echoed back.

Message	Direction	Purpose
`claim`	station → coord	ask before committing a reservation
`renew`	station → coord	re-present every claim held, every 10 s
`release`	station → coord	fire and forget, on cancel, expiry or completion
`who_do_you_hold`	coord → station	the rebuild query of section 8.4
`station_up`	station → coord	announcement, at start-up and every 30 s
`station_stats`	station → coord	free, held and charging counters
`session_closed`	station → coord	a session row was written, relay it to Java

messages are implemented and which are deliberately absent, and give the heartbeat interval and the missed-beat budget, since section 8.5 depends on those numbers.]

9.4 Claims: station to coordinator

Distributed Erlang, with the coordinator a `gen_server` registered locally as `vs_coord_srv` on each coordinator node and addressed as `{vs_coord_srv, Node}`. Calls carry a 2000 ms timeout, and a timeout is treated exactly like an unreachable node.

9.4.1 Acquire

```
{claim, ReqId, VehicleId, UserId, StationId, ConnId}

{ok, ReqId, ClaimId, GrantedAt, ExpiresAt}
{error, ReqId, already_held | suspended | rebuilding | unknown_station}
{not_serving, LeaderNode | undefined}
```

Listing 2: Request and the four possible replies.

Each refusal means something different to the driver, so the station maps them apart: `already_held` says this car is committed at another station, `suspended` says the account is serving a no-show penalty, and `rebuilding` is temporary and produces a retry rather than an error on screen. `unknown_station` means the coordinator has not heard the announcement yet, so the station re-announces itself and tries once more.

Two properties of the reply are load-bearing elsewhere in this document. `ExpiresAt` is always longer than the reservation lease, by `CLAIM_GRACE_SECONDS`, so a claim never expires underneath a reservation that is still alive. And `GrantedAt` is issued by the coordinator, never by the station, for the reason given in section 7.2: the station stores it and echoes it back, so that *oldest wins* always compares two timestamps that came from the same clock.

9.4.2 Renew, and why it carries more than it seems

A station renews all its claims in one message every ten seconds, and the reply names those still valid and those revoked.

```
{renew, StationId, [{ClaimId, VehicleId, ConnId, UserId, GrantedAt}]}
{renewed, Ok, Revoked, NewExpiresAt}
```

Every renewal carries `GrantedAt` and `UserId`, including the hundredth one for a claim nothing has disturbed. The reason is recovery: a renewal is how a newly elected leader learns about claims it never granted. A claim it has never seen is **accepted** and adopted with the

timestamp the station reports, which is what makes section 8.4 converge even when the rebuild query reaches nobody. Carrying `UserId` lets that leader enforce a suspension immediately, instead of holding a claim it cannot attribute to an account.

A revoked claim is not an error to be logged. The station must cancel the reservation or stop the session, free the connector, and tell the driver why. Revocation is how the system converges after a conflict, and the station obeys it even when it believes its claim is valid.

9.4.3 Finding the leader

The station keeps its idea of the current leader in `vs_claim_client`, starting from the first entry of `COORD_NODES`. A `not_serving` carrying a node updates it and the call is retried once. A `not_serving` without one, a timeout or a `noproc` makes it walk the configured list round-robin, for at most one full pass.

If the pass fails, the station gives up and refuses the reservation. It does not queue the request and it never blocks the connector process, which is the rule that keeps a coordination outage from touching cars that are already charging: no claim means no new reservation, and walk-in charging on a free connector needs no claim at all.

9.4.4 Behavioural rules

Binding on both implementations, and each one is a guarantee something else in this document depends on:

1. **Claim first, commit after.** A connector never moves to `held` before the `ok` arrives.
2. **No claim, no reservation.** An unreachable coordinator means refusals.
3. **The coordinator serves only as leader and in quorum.** In any other state it answers `not_serving` or `rebuilding`, never a grant.
4. **Revocation is authoritative.**
5. **Oldest wins**, by `GrantedAt`.
6. **Releases are best-effort.** Correctness never depends on one arriving, because expiry is the backstop.

9.5 The JInterface bridge: coordinator to back office

The back office does not poll the cluster. It joins it. Java opens an `OtpNode` named `voltshare_bo`, registers a mailbox called `backoffice`, and receives Erlang terms as the coordinator produces them. Every page that lists stations then reads from memory.

Java is a **hidden node**: it takes part in the distribution protocol and is addressable by name, but it does not appear in `nodes()` and takes no part in the quorum. That is the correct arrangement, because the back office must never influence who is elected leader. The bridge runs on its own daemon thread and reconnects every three seconds, so a coordinator restart never requires restarting Tomcat.

A detail that cost a day. The `jinterface` artifact published on Maven Central is version 1.6.1, from 2011, and it cannot complete the distribution handshake with an OTP 29 node: `OtpNode.ping` returns false with no exception to point at the cause. The jar we ship is built from the sources of the OTP release we run, and it has to move whenever the OTP version in `Dockerfile.erlang` moves.

`stations_update` is always a complete snapshot. `StationDirectory` replaces its whole list atomically, because a partial update would leave a station on the page after its node had gone down. The WebSocket URL inside it is produced by the coordinator and never assembled by Java: the back office does not know the deployment topology, and publishing a reachable

Table 6: Messages on the bridge.

Message	Direction	Purpose
`stations_update`	coord → Java	the complete station list, on change and every 30 s
`leader`	coord → Java	who is serving now
`session_closed`	coord → Java	a session was written; price it now
`no_show`, `show_up`	coord → Java	penalty accounting
`notify`	coord → Java	something to tell a driver next time they look
`get_stations`	Java → coord	asked once on connect, so no page starts empty
`user_suspended`	Java → coord	a no-show counter tripped
`user_unsuspended`	Java → coord	the penalty expired

address is the cluster's job. A malformed entry is logged and skipped, so one broken station cannot blank the directory.

Suspensions recover by pushing. The suspended set lives in the coordinator's memory while the counter behind it lives in MySQL, so a coordinator that has just started serving knows nothing about it. It cannot ask the stations either, because no station holds a suspension. The back office therefore repeats every running suspension on each `leader` announcement it hears, and the serving coordinator republishes that announcement every 30 seconds. A coordinator that restarts and wins again converges without anyone asking a question.

This is the asymmetry worth noticing, and section 8.3 states it in one line: claims are recovered by *asking*, because the stations own them, and suspensions by *pushing*, because nobody in the cluster does. State is recovered from whoever owns it, in whichever direction that turns out to be.

What happens when the bridge fails. No coordinator reachable at start-up means retries every three seconds, and pages that show an empty list with a warning: a stale list presented as current would be worse than none. A coordinator dying while Tomcat runs leaves the last snapshot in memory, marked stale after 90 seconds. Tomcat dying leaves the cluster untouched, because the back office sits on the path of no reservation and no session.

10 Deployment

[to be written — 2 pages. Seven nodes, the containers, the networks, how the system is started. Taken from deploy/docker-compose.yml, not from memory. State precisely that the deployment runs on one host with seven Erlang and JVM nodes, and that a genuine network partition is produced with `docker network disconnect` rather than with a second machine. Include the per-site networks, which exist to make a site partition demonstrable. Sources: deploy/, DEMO.md.]

11 Testing and Demonstration

[to be written — 2 pages. What is tested automatically (EUnit, scripts/eunit_check.sh, with the expected test count) and what is shown live. The scenarios are in DEMO.md: contention on one connector, cross-node claim, no-show, power sharing in real time, coordinator failover, station crash, station partition. For each one: what appears on screen and which problem of section 5 it demonstrates. That mapping is what turns a demo into evidence.]

12 Future Works

[to be written — 1 page. Site power shared across stations on the same grid connection; partitioning the vehicle space across coordinators; operator dashboard with the ability to take connectors out of service; native push notifications, which the overstay notice needs. Short and honest: these are things not done, not promises. Sources: SCOPE §10, DESIGN-NOTES §7.]